

TASK 19: ASSIGNMENT NO 19

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repository Link: <https://github.com/anujaw193-star/python-project-collection>

1.

- **Dataset Name:** House Prices – Advanced Regression Techniques.
 - **Target Variable:** SalePrice
 - **Type of Target Variable:** Continuous (Numerical)
 - **Machine Learning Problem:** Regression
-

2.

Code & Output:

```
[E1] import numpy as np
import pandas as pd

[E2] # Load the dataset
df = pd.read_csv("train (1).csv")
# Display first 10 rows
df.head(10)
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	PoolArea	PoolQC	Fence	MiscFeature	MiscV
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	0	NaN	NaN	NaN	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	0	NaN	NaN	NaN	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	0	NaN	NaN	NaN	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	0	NaN	NaN	NaN	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	0	NaN	NaN	NaN	
5	6	50	RL	85.0	14115	Pave	NaN	IR1	Lvl	AllPub	0	NaN	MnPrv	Shed	7
6	7	20	RL	75.0	10094	Pave	NaN	Reg	Lvl	AllPub	0	NaN	NaN	NaN	
7	8	60	RL	NaN	10352	Pave	NaN	IR1	Lvl	AllPub	0	NaN	NaN	Shed	3
8	9	50	RM	51.0	6120	Pave	NaN	Reg	Lvl	AllPub	0	NaN	NaN	NaN	
9	10	190	RL	50.0	7420	Pave	NaN	Reg	Lvl	AllPub	0	NaN	NaN	NaN	

10 rows x 16 columns

Explanation:

I selected the House Prices - Advanced Regression Techniques dataset because it is a widely used dataset for regression problems. It contains various features of houses, such as size, location, year built, number of rooms, and garage information. The dataset is suitable for exploring data preprocessing, visualization, and machine learning techniques. The target variable is SalePrice, which is a continuous numerical value representing the selling price of a house. The objective of this project is to build a regression model that can accurately predict the SalePrice based on the given house features. This dataset is ideal for learning real-world regression analysis and predictive modeling.

3.

Code:

```
[4]  # Check Missing Values
missing_values = df.isnull().sum()
# Missing Values Percentage
missing_percentage = (missing_values / len(df)) * 100
# Create a summary table
missing_df = pd.DataFrame({
    "Missing Values": missing_values,
    "Percentage": missing_percentage
})
# Display only columns having missing values
missing_df[missing_df["Missing Values"] > 0]
print("Missing Values Summary:")
print(missing_df.sort_values(by="Percentage", ascending=False))

# Handle Missing Values
# Numerical columns -> Fill with Median
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
for col in num_cols:
    df[col] = df[col].fillna(df[col].median())
# Categorical columns -> Fill with Mode
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
# Verify missing values
print("\nRemaining Missing Values:")
print(df.isnull().sum())
# Check Duplicate Rows
duplicates = df.duplicated().sum()
print("\nDuplicate Rows:", duplicates)
# Remove duplicates
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)
```

Output:

```
... Missing Values Summary:
      Missing Values  Percentage
Id                  0          0.0
MSSubClass          0          0.0
MSZoning            0          0.0
LotFrontage        0          0.0
LotArea            0          0.0
...                ...          ...
MoSold             0          0.0
YrSold            0          0.0
SaleType          0          0.0
SaleCondition     0          0.0
SalePrice         0          0.0

[81 rows x 2 columns]

Remaining Missing Values:
Id          0
MSSubClass  0
MSZoning    0
LotFrontage 0
LotArea     0
..
MoSold      0
YrSold      0
SaleType    0
SaleCondition 0
SalePrice   0
Length: 81, dtype: int64

Duplicate Rows: 0
Shape after removing duplicates: (1460, 81)
```

Observations and Steps Taken:

- i. Checked the dataset for missing values in each column and calculated the percentage of missing values.

- ii. Numerical columns were filled using the median, which is less affected by outliers.
- iii. Categorical columns were filled using the mode, as it represents the most frequent category.
- iv. After handling missing values, the dataset was checked again to ensure no missing values remained.
- v. Duplicate rows were identified using duplicated() and removed using drop_duplicates().
- vi. The cleaned dataset is now ready for further exploratory data analysis (EDA) and regression model building.

4.

Code:

```
[5] ✓ Os # Statistical Summary of Numeric Columns
print("Statistical Summary:")
print(df.describe())

# Target Variable Statistics
print("\nTarget Variable: SalePrice")

print("Minimum SalePrice :", df["SalePrice"].min())
print("Maximum SalePrice :", df["SalePrice"].max())
print("Mean SalePrice :", df["SalePrice"].mean())
print("Median SalePrice :", df["SalePrice"].median())
```

Output:

...	Statistical Summary:						
	Id	MSSubClass	LotFrontage	LotArea	OverallQual		\
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000		
mean	730.500000	56.897260	69.863699	10516.828082	6.099315		
std	421.610009	42.300571	22.027677	9981.264932	1.382997		
min	1.000000	20.000000	21.000000	1300.000000	1.000000		
25%	365.750000	20.000000	60.000000	7553.500000	5.000000		
50%	730.500000	50.000000	69.000000	9478.500000	6.000000		
75%	1095.250000	70.000000	79.000000	11601.500000	7.000000		
max	1460.000000	190.000000	313.000000	215245.000000	10.000000		
	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	...	\
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	...	
mean	5.575342	1971.267808	1984.865753	103.117123	443.639726	...	
std	1.112799	30.202904	20.645407	180.731373	456.098091	...	
min	1.000000	1872.000000	1950.000000	0.000000	0.000000	...	
25%	5.000000	1954.000000	1967.000000	0.000000	0.000000	...	
50%	5.000000	1973.000000	1994.000000	0.000000	383.500000	...	
75%	6.000000	2000.000000	2004.000000	164.250000	712.250000	...	
max	9.000000	2010.000000	2010.000000	1600.000000	5644.000000	...	
	WoodDeckSF	OpenPorchSF	EnclosedPorch	3SsnPorch	ScreenPorch		\
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000		
mean	94.244521	46.660274	21.954110	3.409589	15.060959		
std	125.338794	66.256028	61.119149	29.317331	55.757415		
min	0.000000	0.000000	0.000000	0.000000	0.000000		
25%	0.000000	0.000000	0.000000	0.000000	0.000000		
50%	0.000000	25.000000	0.000000	0.000000	0.000000		
75%	168.000000	68.000000	0.000000	0.000000	0.000000		
max	857.000000	547.000000	552.000000	508.000000	480.000000		

```

max      857.000000    547.000000    552.000000    508.000000    480.000000
...
count      1460.000000    1460.000000    1460.000000    1460.000000    1460.000000
mean        2.758904     43.489041     6.321918    2007.815753    180921.195890
std         40.177307     496.123024     2.703626     1.328095     79442.502883
min          0.000000      0.000000     1.000000    2006.000000    34900.000000
25%          0.000000      0.000000     5.000000    2007.000000    129975.000000
50%          0.000000      0.000000     6.000000    2008.000000    163000.000000
75%          0.000000      0.000000     8.000000    2009.000000    214000.000000
max         738.000000    15500.000000    12.000000    2010.000000    755000.000000

[8 rows x 38 columns]

Target Variable: SalePrice
Minimum SalePrice : 34900
Maximum SalePrice : 755000
Mean SalePrice : 180921.19589041095
Median SalePrice : <bound method Series.median of 0      208500
1      181500
2      223500
3      140000
4      250000
...
1455    175000
1456    210000
1457    266500
1458    142125
1459    147500
Name: SalePrice, Length: 1460, dtype: int64>

```

Observations :

- i. The SalePrice values vary widely, indicating a large price range among houses.
- ii. The mean SalePrice is higher than the median, which suggests the data is right-skewed due to some very expensive houses.
- iii. There is a significant difference between the minimum and maximum SalePrice, showing high variability in house prices.
- iv. Most houses are priced around the median value, while a few luxury houses increase the average (mean).
- v. The statistical summary (describe()) helps identify the spread of numerical features and can also indicate potential outliers.

5.

Code:

```

[13]
✓ 2s # Load dataset
df = pd.read_csv("train (1).csv")

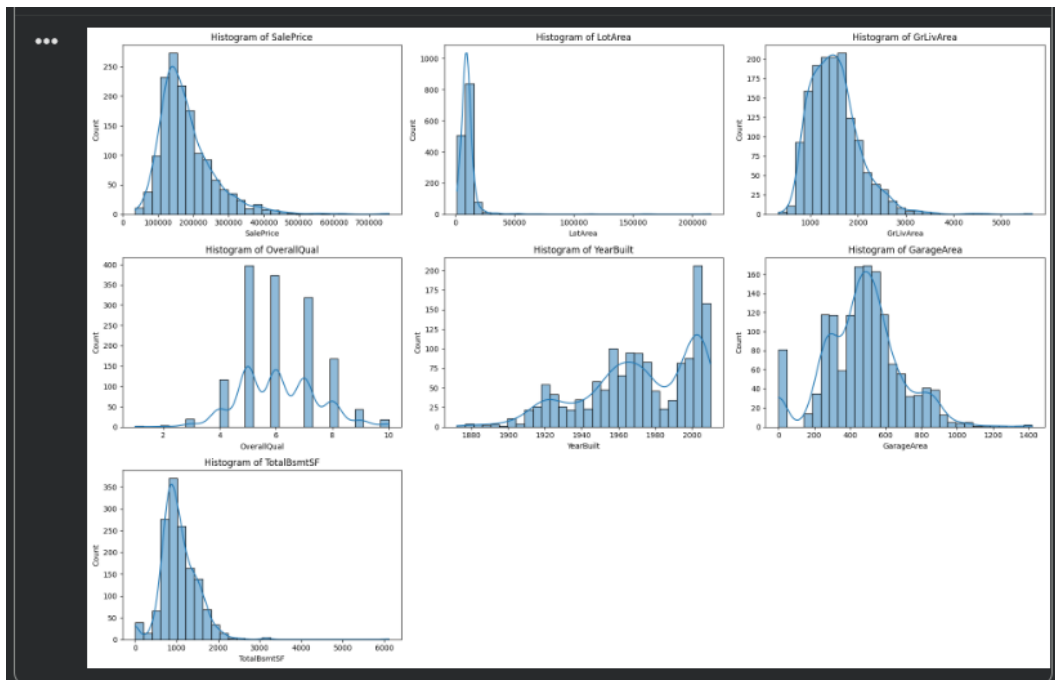
# Important numeric columns
columns = [
    "SalePrice",
    "LotArea",
    "GrLivArea",
    "OverallQual",
    "YearBuilt",
    "GarageArea",
    "TotalBsmntSF",
]

# Plot histograms
plt.figure(figsize=(18,12))

for i, col in enumerate(columns):
    plt.subplot(3,3,i+1)
    sns.histplot(df[col], bins=30, kde=True)
    plt.title(f"Histogram of {col}")
plt.tight_layout()
plt.show()

```

Output;



Detailed:

1. SalePrice (Target Variable)

- The distribution is right-skewed.
- Most houses have medium selling prices.
- A few expensive houses create a long right tail.
- Some high-priced houses may be treated as outliers.

2. LotArea

- Most properties have small to medium-sized plots.
- A few houses have very large land areas.
- The distribution is highly right-skewed.
- Large plots can significantly increase house prices.

3. GrLivArea (Above Ground Living Area)

- Most houses have moderate living space.
- Very large houses are less common.
- The data is slightly right-skewed.
- Houses with larger living areas generally sell for higher prices.

4. OverallQual

- Most houses have quality ratings between 5 and 7.
- Very low and very high quality houses are rare.
- This feature is likely to have a strong impact on SalePrice.
- Better quality usually results in higher selling prices.

5. YearBuilt

- Most houses were built after 1950.
- Very old houses are fewer in number.

- The distribution shows more modern homes than historic ones.
- Newer houses generally have higher market values.

6. GarageArea

- Most garages have medium storage capacity.
- Some houses have very large garages.
- A few houses may have no garage at all.
- Larger garages often add value to a property.

7. TotalBsmtSF

- Basement sizes vary considerably.
- Most houses have medium-sized basements.
- Some properties have very large basements.
- Larger basements may positively influence the selling price.

Overall Observation:

Most numeric features are right-skewed, indicating the presence of high-value observations.

Several variables contain outliers, which may require treatment before model training.

Features such as GrLivArea, OverallQual, GarageArea, and TotalBsmtSF are expected to be strongly related to SalePrice and are important predictors for regression.

6.

Categorical Columns CODE & OUTPUT:

```
[14]
✓ Os # Find categorical columns
cat_cols = df.select_dtypes(include=['object']).columns

print("Categorical Columns:")
print(cat_cols)

print("\nTotal Categorical Columns:", len(cat_cols))

... Categorical Columns:
Index(['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities',
      'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2',
      'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st',
      'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundation',
      'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2',
      'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenQual',
      'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual',
      'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature',
      'SaleType', 'SaleCondition'],
      dtype='object')

Total Categorical Columns: 43
```

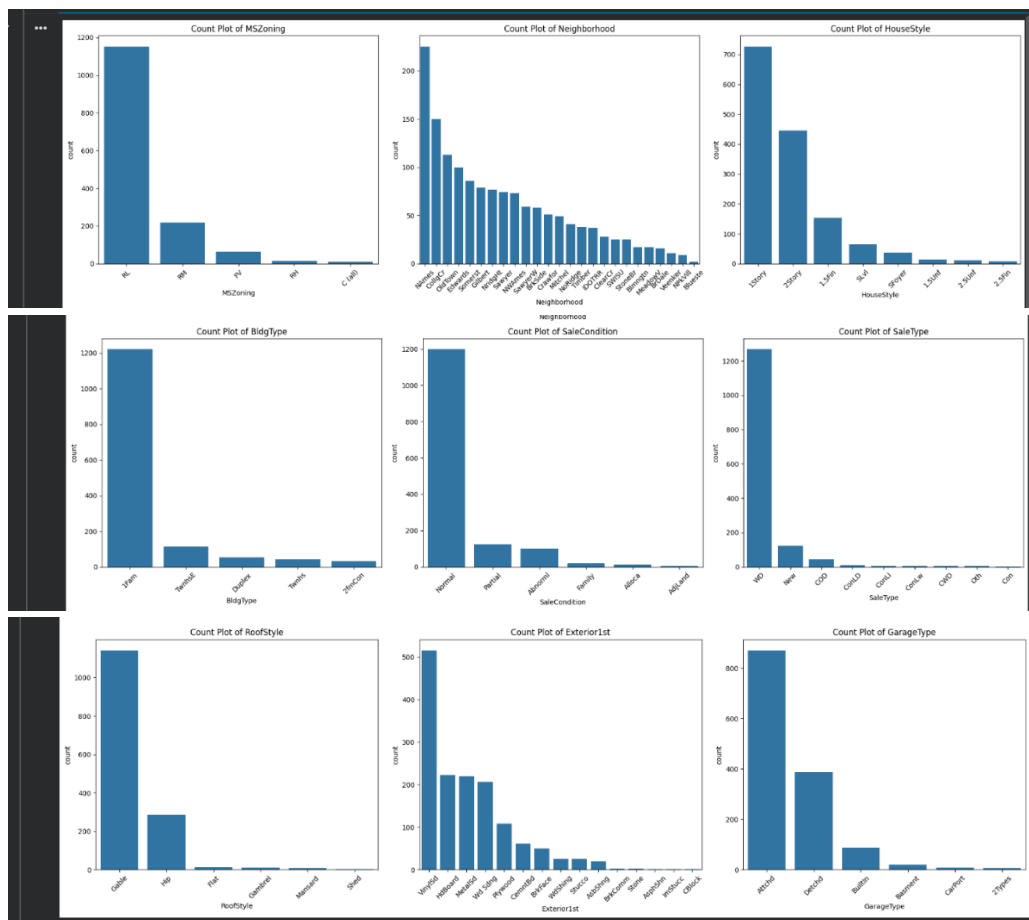
Count plots

Code:

```
[15]
✓ 1s # Import categorical columns
cat_columns = [
    "MSZoning",
    "Neighborhood",
    "HouseStyle",
    "BldgType",
    "SaleCondition",
    "SaleType",
    "RoofStyle",
    "Exterior1st",
    "GarageType",
]
plt.figure(figsize=(20, 18))

for i, col in enumerate(cat_columns):
    plt.subplot(3, 3, i + 1)
    sns.countplot(data=df, x=col, order=df[col].value_counts().index)
    plt.xticks(rotation=45)
    plt.title(f"Count Plot of {col}")
plt.tight_layout()
plt.show()
```

Output:



Business / Domain Insights

1. MSZoning

- Most houses belong to the Residential Low Density (RL) zone.
- This indicates that suburban residential properties dominate the dataset.

2. Neighborhood

- Some neighborhoods have many more houses than others.
- Popular neighborhoods provide more training examples and often have different price levels.

3. HouseStyle

- 1-Story and 2-Story houses are the most common.
- Other styles are relatively rare.

4. BldgType

- Single-family detached houses (1Fam) make up the majority of properties.
- This reflects the dominant housing type in the market.

5. SaleCondition

- Most houses were sold under Normal sale conditions.
- Special sale conditions such as foreclosure or partial sales occur much less frequently.

6. SaleType

- The majority of sales are WD (Warranty Deed) transactions.
- Other sale types represent only a small portion of the market.

7. RoofStyle

- Gable roofs are the most common.
- Other roof styles are much less frequent.

8. Exterior1st

- Vinyl siding is the most frequently used exterior material.
- Brick, wood, and metal exteriors appear less often.

9. GarageType

- Most houses have an Attached Garage.
- Attached garages are generally preferred because they provide convenience and may increase property value.

Overall Observation

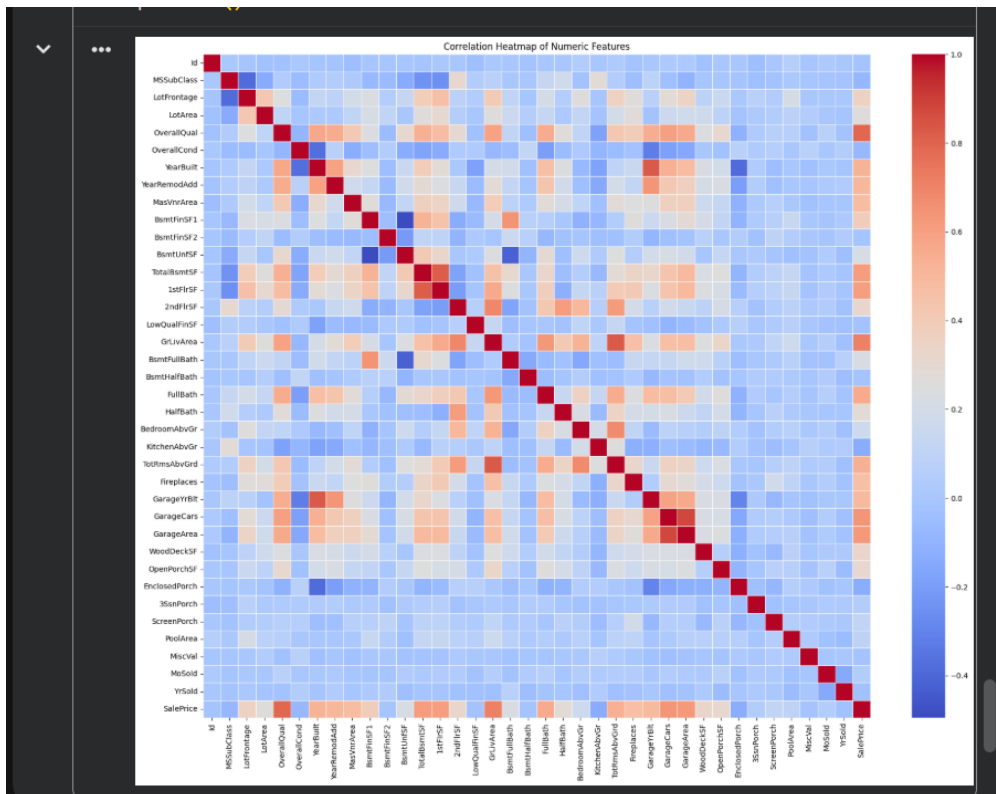
- The dataset is dominated by residential, single-family houses.
- Most properties are sold through normal market transactions.
- Features such as Neighborhood, MSZoning, HouseStyle, and GarageType are likely to influence the SalePrice and are useful predictors for a regression model.

7.

Code:

```
[16] ✓ Os # Select only numeric columns
numric_df = df.select_dtypes(include=['int64', 'float64'])
# Correlation matrix
corr_matrix = numric_df.corr()
# Plot heatmap
plt.figure(figsize=(20, 16))
sns.heatmap(corr_matrix,
            cmap='coolwarm',
            linewidths=0.5)
plt.title("Correlation Heatmap of Numeric Features")
plt.show()
```

Output:



Features Most Correlated with SalePrice

```
[18]
✓ 0s # Correlation with target variable
saleprice_corr = corr_matrix["SalePrice"].sort_values(ascending=False)
print(saleprice_corr)
```

Output;

Feature	Correlation
SalePrice	1.000000
OverallQual	0.790982
GrLivArea	0.708624
GarageCars	0.640409
GarageArea	0.623431
TotalBsmntSF	0.613581
1stFlrSF	0.605852
FullBath	0.560664
TotRmsAbvGrd	0.533723
YearBuilt	0.522897
YearRemodAdd	0.507101
GarageYrBlt	0.486362
MasVnrArea	0.477493
Fireplaces	0.466929
BsmntFinSF1	0.386420
LotFrontage	0.351799
WoodDeckSF	0.324413
2ndFlrSF	0.319334
OpenPorchSF	0.315856
HalfBath	0.284108
LotArea	0.263843
BsmntFullBath	0.227122
BsmntUnfSF	0.214479
BedroomAbvGr	0.168213
ScreenPorch	0.111447
PoolArea	0.092404
MoSold	0.046432
3SsnPorch	0.044584
BsmntFinSF2	-0.011378
BsmntHalfBath	-0.016844
MiscVal	-0.021190

```

Bsm1HalfBath -0.016844
MiscVal -0.021190
Id -0.021917
LowQualFinSF -0.025606
YrSold -0.028923
OverallCond -0.077856
MSSubClass -0.084284
EnclosedPorch -0.128578
KitchenAbvGr -0.135907
Name: SalePrice, dtype: float64

```

Observations

- *OverallQual has the strongest positive correlation with SalePrice, indicating that houses with higher overall quality generally sell for higher prices.*
- *GrLivArea (Above Ground Living Area) is also highly positively correlated with SalePrice. Larger living spaces usually increase a house's selling price.*
- *GarageCars and GarageArea show strong positive correlations with SalePrice, suggesting that houses with larger garages tend to have higher market values.*
- *TotalBsm1SF and 1stFlrSF have positive correlations with SalePrice, meaning houses with larger basements and first-floor areas are generally more expensive.*
- *YearBuilt has a moderate positive correlation with SalePrice, indicating that newer houses often have higher selling prices.*
- *Some features have weak or near-zero correlation with SalePrice, meaning they have little linear influence on the target variable individually.*
- *The heatmap also shows that some predictor variables are highly correlated with each other (for example, GarageCars and GarageArea), which may indicate multicollinearity and should be considered during model building.*

8.

Code:

```

[20]
✓ Os
# Independent Features (X)
X = df.drop("SalePrice", axis=1)
# Dependent Feature (y)
y = df["SalePrice"]
print("Independent Features (X):")
print(X.columns)
print("\nDependent Feature (y):")
print(y.name)

print("\nShape of X:", X.shape)
print("Shape of y:", y.shape)

```

Output:

```

... Independent Features (X):
Index(['Id', 'MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea', 'Street',
      'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig',
      'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType',
      'HouseStyle', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd',
      'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType',
      'MasVnrArea', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual',
      'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1',
      'BsmtFinType2', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'Heating',
      'HeatingQC', 'CentralAir', 'Electrical', '1stFlrSF', '2ndFlrSF',
      'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath',
      'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual',
      'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType',
      'GarageYrBlt', 'GarageFinish', 'GarageCars', 'GarageArea', 'GarageQual',
      'GarageCond', 'PavedDrive', 'WoodDeckSF', 'OpenPorchSF',
      'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'PoolQC',
      'Fence', 'MiscFeature', 'MiscVal', 'MoSold', 'YrSold', 'SaleType',
      'SaleCondition'],
      dtype='object')

Dependent Feature (y):
SalePrice

Shape of X: (1460, 80)
Shape of y: (1460,)

```

Justification:

- *SalePrice is a continuous numerical variable, making it suitable as the target variable for a regression problem.*
- *All remaining columns contain information about house size, location, quality, age, garage, basement, and other characteristics that influence the selling price.*
- *These features serve as independent variables (X) because they help explain and predict the value of SalePrice.*
- *The objective of the model is to learn the relationship between the house features (X) and the selling price (y) so that it can accurately predict the price of new houses.*

9.

Code:

```

[21] ✓ Os # Identify categorical columns
cat_cols = df.select_dtypes(include='object').columns
print("Categorical Columns:")
print(cat_cols)
# Before Encoding
print("\nBefore Encoding:")
print("\nMSZoning:")
print(df["MSZoning"].head())
print("\nNeighborhood:")
print(df["Neighborhood"].head())
# One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True)
# After Encoding
print("\nAfter Encoding (Columns Created):")
print("\nMSZoning Columns:")
print([col for col in df_encoded.columns if "MSZoning" in col])

print("\nNeighborhood Columns:")
print([col for col in df_encoded.columns if "Neighborhood" in col])
# Shape before and after encoding
print("\nOriginal Shape :", df.shape)
print("Encoded Shape :", df_encoded.shape)

```

Output:

```

... Categorical Columns:
Index(['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities',
      'LotConfig', 'LandSlope', 'Neighborhood', 'Condition1', 'Condition2',
      'BldgType', 'HouseStyle', 'RoofStyle', 'RoofMatl', 'Exterior1st',
      'Exterior2nd', 'MasVnrType', 'ExterQual', 'ExterCond', 'Foundation',
      'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2',
      'Heating', 'HeatingQC', 'CentralAir', 'Electrical', 'KitchenQual',
      'Functional', 'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual',
      'GarageCond', 'PavedDrive', 'PoolQC', 'Fence', 'MiscFeature',
      'SaleType', 'SaleCondition'],
      dtype='object')

Before Encoding:

MSZoning:
0    RL
1    RL
2    RL
3    RL
4    RL
Name: MSZoning, dtype: object

Neighborhood:
0    CollgCr
1    Veenker
2    CollgCr
3    Crawfor
4    NoRidge
Name: Neighborhood, dtype: object

After Encoding (Columns Created):

MSZoning Columns:

```

```

MSZoning:
0    RL
1    RL
2    RL
3    RL
4    RL
Name: MSZoning, dtype: object

Neighborhood:
0    CollgCr
1    Veenker
2    CollgCr
3    Crawfor
4    NoRidge
Name: Neighborhood, dtype: object

After Encoding (Columns Created):

MSZoning Columns:
['MSZoning_FV', 'MSZoning_RH', 'MSZoning_RL', 'MSZoning_RM']

Neighborhood Columns:
['Neighborhood_Blueste', 'Neighborhood_BrDale', 'Neighborhood_BrkSide', 'Neighborhood_ClearCr', 'Neighborhood_CollgCr', 'Neighborhood_Crawfor']

Original Shape : (1460, 81)
Encoded Shape : (1460, 246)

```

Explanation :

- The dataset contains many categorical (text) features, which cannot be used directly by machine learning algorithms.
- One-Hot Encoding converts each category into separate binary (0/1) columns, making the data suitable for regression models.
- The parameter `drop_first=True` removes one category from each feature to avoid dummy variable trap (multicollinearity).
- After encoding, all features become numeric, allowing machine learning algorithms to process the dataset efficiently.
- One-Hot Encoding preserves category information without assigning an artificial order, making it the most suitable technique for nominal categorical variables.

10.

Code & Output:

```
[23] # One-Hot Encoding
df_encoded = pd.get_dummies(df, drop_first=True)
# Independent Features (X)
X = df_encoded.drop("SalePrice", axis=1)
# Dependent Feature (y)
y = df_encoded["SalePrice"]
# Standard Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Convert scaled data into DataFrame
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
# Display first 5 rows
X_scaled.head()
```

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	...	SaleType_ConLI	S
0	-1.730865	0.073375	-0.208034	-0.207142	0.651479	-0.517200	1.050994	0.878668	0.510015	0.575425	...	-0.058621	
1	-1.728492	-0.872563	0.408895	-0.091886	-0.071836	2.179628	0.156734	-0.429577	-0.572835	1.171992	...	-0.058621	
2	-1.726120	0.073375	-0.084449	0.073480	0.651479	-0.517200	0.984752	0.830215	0.322174	0.092907	...	-0.058621	
3	-1.723747	0.309859	-0.414011	-0.096887	0.651479	-0.517200	-1.863632	-0.720298	-0.572835	-0.498274	...	-0.058621	
4	-1.721374	0.073375	0.574676	0.375148	1.374795	-0.517200	0.951632	0.733308	1.360826	0.463568	...	-0.058621	

5 rows x 245 columns

Explanation

- *StandardScaler standardizes all independent features by transforming them to have a mean of 0 and a standard deviation of 1.*
 - *Feature scaling ensures that variables with large values do not dominate those with smaller values.*
 - *Scaling improves the performance of many machine learning algorithms, especially Linear Regression, Logistic Regression, SVM, KNN, and Neural Networks.*
 - *The target variable SalePrice is not scaled because it is the value we want the model to predict.*
-